Title

Adam Abramowitz Max Desantis Isaac Dreeben Abhi Mummaneni

Abstract

1 Introduction

2 Related Work

2.1 Early Methods in Graph Learning

Early graph-learning methods were motivated by the observation that graph structure can be used as an inductive bias for representation learning. In Laplacian Eigenmaps, Belkin and Niyogi [2] construct a weighted graph over data points and seek low-dimensional embeddings that keep neighboring nodes close. Let W be the weighted adjacency matrix, D the diagonal degree matrix, and $L = D - W$ the combinatorial graph Laplacian. The embedding matrix $Y \in \mathbb{R}^{|V| \times d}$ is obtained by solving

$$\min_{Y} \ \frac{1}{2} \sum_{i,j} W_{ij} \|Y_i - Y_j\|_2^2 = \min_{Y} \ \operatorname{tr}(Y^\top L Y), \qquad \text{s.t. } Y^\top D Y = I, \ Y^\top D \mathbf{1} = 0.$$

This objective makes the geometric assumption explicit: if two examples are adjacent in the graph, their learned representations should vary smoothly across that edge. The solution is given by the generalized eigenvectors of $Ly = \lambda Dy$, connecting graph embeddings to spectral approximations of the Laplace–Beltrami operator on an underlying manifold.

This smoothness viewpoint was further developed in manifold regularization [3], where the graph is used to regularize functions defined on data points. For a graph signal $f : V \to \mathbb{R}$, the graph smoothness penalty is

$$\mathcal{R}_G(f) = \frac{1}{2} \sum_{i,j} W_{ij} \big(f_i - f_j\big)^2 = f^\top L f.$$

Thus, learning on graphs can be interpreted as learning functions whose values change slowly across high-weight edges. Shuman et al. [16] generalized this interpretation through the language of graph signal processing. If $L = U \Lambda U^\top$ is an eigendecomposition of the graph Laplacian, then the graph Fourier transform of a signal $x \in \mathbb{R}^{|V|}$ is

$$\widehat{x} = U^\top x, \qquad x = U \widehat{x}.$$

Eigenvectors associated with small eigenvalues correspond to smooth, low-frequency variation over the graph, while larger eigenvalues correspond to high-frequency oscillations. This spectral viewpoint provides the foundation for defining graph convolutional filters.

2.2 Graph Convolutional Networks

Spectral graph neural networks extend convolution from Euclidean domains to irregular graph domains by filtering graph signals in the Laplacian eigenbasis [4, 5, 12]. Given a signal x and a spectral filter g_θ, convolution can be written as

$$g_\theta * x = U g_\theta(\Lambda) U^\top x,$$

where $g_\theta(\Lambda)$ scales each graph-frequency component independently. This formulation is expressive but depends on the eigendecomposition of the graph Laplacian, which is expensive for large graphs and not naturally transferable across graphs with different eigenbases.

Chebyshev spectral networks address this limitation by approximating the filter as a polynomial of the rescaled Laplacian [5]:

$$g_\theta * x \approx \sum_{k=0}^{K} \theta_k T_k(\widetilde{L}) x, \qquad \widetilde{L} = \frac{2}{\lambda_{\max}} L - I,$$

where T_k denotes the k-th Chebyshev polynomial. Because this filter is a K-th order polynomial in L, it is localized to K-hop neighborhoods and avoids explicit eigendecomposition.

Kipf and Welling [12] further simplified this construction by using a first-order approximation and a renormalized adjacency matrix. With self-loops $\widetilde{A} = A + I$ and degree matrix $\widetilde{D}_{ii} = \sum_j \widetilde{A}_{ij}$, a standard GCN layer is

$$H^{(\ell+1)} = \sigma\Big(\widetilde{D}^{-1/2} \widetilde{A} \widetilde{D}^{-1/2} H^{(\ell)} W^{(\ell)}\Big).$$

This layer averages transformed node features over local neighborhoods, making GCNs an efficient and widely used architecture for semi-supervised node classification. However, repeated applications of the propagation operator can also cause node representations to become indistinguishable. Li, Han, and Wu [13] interpret GCN propagation as Laplacian smoothing; after K propagation steps, the representation is dominated by powers of the normalized adjacency:

$$H^{(K)} \approx \Big(\widetilde{D}^{-1/2} \widetilde{A} \widetilde{D}^{-1/2}\Big)^K X.$$

This smoothing perspective motivates later work on deeper GNNs, residual connections, attention mechanisms, and architectures designed to preserve long-range or high-frequency information.

2.3 Graph Autoencoders and Generative Graph Models

A second line of work relevant to graph representation learning comes from autoencoders and variational autoencoders. The variational autoencoder framework of Kingma and Welling [10] introduces a latent-variable model in which an encoder approximates the posterior distribution over latent variables and a decoder reconstructs the observed data. For data x, latent variable z, prior $p(z)$, encoder $q_\phi(z \mid x)$, and decoder $p_\theta(x \mid z)$, the evidence lower bound is

$$\mathcal{L}_{\mathrm{VAE}}(\theta, \phi; x) = \mathbb{E}_{q_\phi(z|x)}\big[\log p_\theta(x \mid z)\big] - \mathrm{KL}(q_\phi(z \mid x) \,\|\, p(z)).$$

The first term encourages accurate reconstruction, while the KL term regularizes the latent space toward the prior. This framework naturally supports probabilistic generation by sampling from $p(z)$ and decoding.

Variational Graph Autoencoders (VGAEs) adapt this idea to graph-structured data by using a GCN encoder and an edge-probability decoder [11]. Given node features X and adjacency matrix A, the encoder factorizes over nodes as

$$q_\phi(Z \mid X, A) = \prod_{i=1}^{|V|} q_\phi(z_i \mid X, A), \qquad q_\phi(z_i \mid X, A) = \mathcal{N}\big(z_i \mid \mu_i, \mathrm{diag}(\sigma_i^2)\big)\,,$$

and the decoder predicts edges through an inner product:

$$p_\theta(A_{ij} = 1 \mid z_i, z_j) = \sigma(z_i^\top z_j).$$

This model connects graph representation learning with link prediction: the latent embedding must encode enough structural information to reconstruct the observed adjacency matrix.

Subsequent generative graph models extend VGAEs from link prediction toward whole-graph generation. GraphVAE [17] decodes latent variables into probabilistic node and edge tensors for small graphs, while Graphite [8] improves the decoder by iteratively refining a generated graph. These models can be viewed as learning a distribution over graphs,

$$p_\theta(G) = \int p_\theta(G \mid z) p(z)\, dz,$$

where the decoder must capture both local edge dependencies and global graph-level constraints. This generative perspective complements discriminative GNNs by emphasizing reconstruction, latent structure, and graph synthesis.

2.4 Relational Graphs and Relational GCNs

Many real-world graphs are not homogeneous: edges may represent different relation types, and nodes may belong to different semantic categories. Relational GCNs extend the GCN update rule to multi-relational graphs by assigning relation-specific transformations [15]. For relation set $\mathcal{R}$, relation-specific neighborhood $\mathcal{N}_i^r$, and normalization constant $c_{i,r}$, the R-GCN update is

$$h_i^{(\ell+1)} = \sigma\left(\sum_{r\in\mathcal{R}} \sum_{j\in\mathcal{N}_i^r} \frac{1}{c_{i,r}} W_r^{(\ell)} h_j^{(\ell)} + W_0^{(\ell)} h_i^{(\ell)}\right).$$

This formulation separates messages by relation type, making it suitable for knowledge graphs and other structured domains in which edge labels carry semantic meaning.

Heterogeneous graph models further generalize this idea by allowing both node-type-specific and edge-type-specific transformations. The Heterogeneous Graph Transformer of Hu et al. [9] uses attention mechanisms whose parameters depend on the source node type, target node type, and relation type. A simplified relation-aware attention score can be written as

$$\alpha_{ij}^r = \mathrm{softmax}_{j\in\mathcal{N}_i^r}\left(\frac{(Q_{\tau(i)} h_i)^\top R_r (K_{\tau(j)} h_j)}{\sqrt{d}}\right),$$

where $\tau(i)$ denotes the type of node i and r denotes the relation connecting nodes i and j. This line of work shows how similarity graphs used in early spectral methods can be extended to typed, relational, and semantically structured graphs.

2.5 Message Passing, Attention, and Graph Transformers

Message passing provides a unifying abstraction for many GNN architectures. Gilmer et al. [7] formulate neural message passing in terms of a message function, aggregation operation, and update function. At layer t, a generic message passing update is

$$m_i^{(t+1)} = \sum_{j \in \mathcal{N}(i)} M_t\Big(h_i^{(t)}, h_j^{(t)}, e_{ij}\Big), \qquad h_i^{(t+1)} = U_t\Big(h_i^{(t)}, m_i^{(t+1)}\Big).$$

GCNs, R-GCNs, and many spectral approximations can be interpreted as instances of this template with different choices of message function, normalization, and aggregation.

Graph Attention Networks make the connection between message passing and attention explicit by replacing fixed neighborhood averaging with learned attention weights [19]. For neighboring nodes i and j, attention coefficients are computed as

$$\alpha_{ij} = \mathrm{softmax}_{j \in \mathcal{N}(i)}\Big(\mathrm{LeakyReLU}\Big(a^\top [Wh_i \,\|\, Wh_j]\Big)\Big),$$

and the node update becomes

$$h_i' = \sigma\left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} W h_j\right).$$

Attention allows the model to assign different weights to different neighbors, rather than assuming that all adjacent nodes contribute equally.

Transformer-style graph models generalize this principle by combining global or constrained attention with structural encodings. Dwivedi and Bresson [6] adapt Transformers to arbitrary graphs using graph-aware positional encodings, while Graphormer [21] demonstrates that full attention can be effective when enriched with structural biases such as shortest-path distances and edge encodings. A common way to express graph-biased attention is

$$\mathrm{Attn}(Q, K, V; B) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d}} + B\right) V,$$

where B_{ij} encodes graph structure. In constrained attention, one may set

$$B_{ij} = \begin{cases} 0, & (i,j) \in E \text{ or } i = j, \\ -\infty, & \text{otherwise}, \end{cases}$$

which recovers neighborhood-restricted attention; richer choices of B permit long-range interactions while preserving graph inductive bias. Recent graph language models [14] further connect this line of work to language modeling by incorporating graph structure into pretrained, Transformer-like architectures.

2.6 Expressivity and Limitations

Although message passing architectures are broadly useful, their expressivity is limited. Xu et al. [20] relate the discriminative power of GNNs to the Weisfeiler–Lehman (WL) graph isomorphism test. In WL-style refinement, node colors are updated by hashing the previous color together with the multiset of neighboring colors:

$$c_i^{(k)} = \mathrm{HASH}\Big(c_i^{(k-1)}, \Big\{c_j^{(k-1)} : j \in \mathcal{N}(i)\Big\}\Big).$$

Similarly, standard message passing GNNs update node embeddings by aggregating multisets of neighbor representations. As a result, many GNNs cannot distinguish graph structures that the 1-WL test cannot distinguish.

A separate limitation is over-squashing: information from a large receptive field must be compressed into fixed-dimensional node embeddings. After T message passing layers, a node representation can depend on all nodes within T hops,

$$h_i^{(T)} = \Phi_T(\{x_j : \mathrm{dist}(i,j) \le T\}) \in \mathbb{R}^d,$$

but the dimension d remains fixed even when the number of contributing nodes grows exponentially with distance. Alon and Yahav [1] identify this bottleneck as a practical obstacle for long-range reasoning on graphs.

These limitations motivate architectures that augment message passing with virtual edges, positional encodings, higher-order structures, or global attention. However, as seen in [18] many approaches that appear to go beyond message passing can be reframed as message passing over an augmented graph,

$$G' = (V', E'), \qquad E \subseteq E',$$

where added edges, nodes, or features change the computational graph over which information flows. This perspective ties together spectral embeddings, GCNs, relational models, graph autoencoders, and graph Transformers: each can be understood as making a different choice about the graph structure, message function, and inductive bias used to propagate information.

3 Results

4 Discussion

5 Conclusion

6 Appendix

References

[1] Uri Alon and Eran Yahav. On the bottleneck of graph neural networks and its practical implications. In *International Conference on Learning Representations*, 2021.

[2] Mikhail Belkin and Partha Niyogi. Laplacian eigenmaps and spectral techniques for embedding and clustering. In *Advances in Neural Information Processing Systems*, volume 14, 2001.

[3] Mikhail Belkin, Partha Niyogi, and Vikas Sindhwani. Manifold regularization: A geometric framework for learning from labeled and unlabeled examples. *Journal of Machine Learning Research*, 7:2399–2434, 2006.

[4] Joan Bruna, Wojciech Zaremba, Arthur Szlam, and Yann LeCun. Spectral networks and locally connected networks on graphs. In *International Conference on Learning Representations*, 2014.

[5] Michaël Defferrard, Xavier Bresson, and Pierre Vandergheynst. Convolutional neural networks on graphs with fast localized spectral filtering. In *Advances in Neural Information Processing Systems*, volume 29, 2016.

[6] Vijay Prakash Dwivedi and Xavier Bresson. A generalization of transformer networks to graphs. *arXiv preprint arXiv:2012.09699*, 2020.

[7] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1263–1272. PMLR, 2017.

[8] Aditya Grover, Aaron Zweig, and Stefano Ermon. Graphite: Iterative generative modeling of graphs. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2434–2444. PMLR, 2019.

[9] Ziniu Hu, Yuxiao Dong, Kuansan Wang, and Yizhou Sun. Heterogeneous graph transformer. In *Proceedings of The Web Conference 2020*, pages 2704–2710. Association for Computing Machinery, 2020.

[10] Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. In *International Conference on Learning Representations*, 2014.

[11] Thomas N. Kipf and Max Welling. Variational graph auto-encoders. *arXiv preprint arXiv:1611.07308*, 2016.

[12] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017.

[13] Qimai Li, Zhichao Han, and Xiao-Ming Wu. Deeper insights into graph convolutional networks for semi-supervised learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.

[14] Moritz Plenz and Anette Frank. Graph language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4477–4494, Bangkok, Thailand, 2024. Association for Computational Linguistics.

[15] Michael Schlichtkrull, Thomas N. Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, and Max Welling. Modeling relational data with graph convolutional networks. In *The Semantic Web*, volume 10843 of *Lecture Notes in Computer Science*, pages 593–607. Springer, 2018.

[16] David I. Shuman, Sunil K. Narang, Pascal Frossard, Antonio Ortega, and Pierre Vandergheynst. The emerging field of signal processing on graphs: Extending high-dimensional data analysis to networks and other irregular domains. *IEEE Signal Processing Magazine*, 30(3):83–98, 2013.

[17] Martin Simonovsky and Nikos Komodakis. GraphVAE: Towards generation of small graphs using variational autoencoders. In *Artificial Neural Networks and Machine Learning – ICANN 2018*, volume 11139 of *Lecture Notes in Computer Science*, pages 412–422. Springer, 2018.

[18] Petar Veličković. Message passing all the way up. In *ICLR 2022 Workshop on Geometrical and Topological Representation Learning*, 2022.

[19] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations*, 2018.

[20] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations*, 2019.

[21] Chengxuan Ying, Tianle Cai, Shengjie Luo, Shuxin Zheng, Guolin Ke, Di He, Yanming Shen, and Tie-Yan Liu. Do transformers really perform bad for graph representation? In *Advances in Neural Information Processing Systems*, volume 34, pages 28877–28888, 2021.